DS7 · Datos a escala

DuckDB · Polars · Parquet · dbt · Spark · Airflow · Kafka

AI Surfer · N3 Data Science

Por qué este módulo

- Las ofertas de ML Engineer y DS senior piden **Spark, Kafka, Airflow, dbt**
- Un modelo es tan bueno como los datos que le llegan
- Al terminar: un **pipeline de datos** para tu portafolio
- Caso: pedidos diarios de **TiendaSur**

Mapa del módulo

1. Cuándo pandas deja de bastar
2. DuckDB y Polars
3. Parquet y particionado
4. Almacén: estrella y capas
5. dbt
6. Spark
7. Orquestación
8. Streaming

7.1 · La escalera de herramientas

Datos	Herramienta
Caben en memoria	pandas
Caben en un disco	**DuckDB / Polars**
No caben en una máquina	**Spark**
Llegan sin parar	**Streaming**

> Sube un peldaño cuando **midas** que lo necesitas

7.2 · Un benchmark honesto

1. **Mismo resultado** (compruébalo)
2. Mismas condiciones
3. Varias repeticiones
4. Mide todo, incluida la lectura
5. Tiempo, memoria y legibilidad
6. No generalices

4 millones de filas: Polars y DuckDB, varias veces más rápidos que pandas

7.2 · Polars lazy

```
(pl.scan_parquet("pedidos/**/*.parquet", hive_partitioning=True)
   .filter(pl.col("estado") == "entregado")
   .group_by("mes")
   .agg((pl.col("cantidad") * pl.col("precio_unitario")).sum())
   .collect())
```

Plan → optimización → ejecución paralela

7.2 · DuckDB

```
duckdb.sql("""
  SELECT mes, SUM(cantidad * precio_unitario)
  FROM read_parquet('pedidos/**/*.parquet')
  WHERE estado = 'entregado' GROUP BY mes
""").df()
```

SQL + sin servidor + lee Parquet directamente

7.3 · Fila frente a columna

CSV	Parquet
texto, por filas	binario, por columnas
lees todo	lees **solo las columnas pedidas**
33 MB	10 MB (mismas 500.000 filas)

- estadísticas por bloque → se saltan bloques

7.3 · Particionado

```
pedidos/
  mes=2025-10/…
  mes=2025-11/…
  …
  mes=2026-09/…
```

`WHERE mes = '2026-09'` → abre **1 de 12** carpetas

Columna **filtrada a menudo** y de **baja cardinalidad**

7.4 · Esquema en estrella

- **Hecho:** `fct_ventas_dia` (día × producto × canal)
- **Dimensiones:** `dim_producto`, fecha, canal…
- **Primero el grano**: ¿qué es una fila?
- Mezclar granos = sumar dos veces

7.4 · Bronce → plata → oro

Bronce	Plata	Oro
copia fiel + metadatos	limpio y tipado	para negocio
nunca se modifica	rechazos con motivo	hechos y dimensiones

Idempotente: borrar el día e insertar el día

7.5 · dbt

- SQL en archivos `.sql` + `{{ ref() }}` → grafo de dependencias
- Transforma **dentro** del almacén (ELT)
- **Tests de datos:** `unique`, `not_null`, `accepted_values`, `relationships`
- `dbt build` = modelos + tests, en orden

7.5 · Un test que falla

```
FAIL 1 accepted_values_fct_ventas_dia_canal
Done. PASS=8 ERROR=1 SKIP=…
```

Código de salida ≠ 0 → el orquestador **para**

Un dato malo no llega al dashboard

7.6 · Spark en una diapositiva

- Datos en **particiones** repartidas en un clúster
- Transformaciones **perezosas**; las **acciones** ejecutan
- `groupBy` / `join` → **shuffle** (lo caro)
- API ≈ pandas + SQL

Practicar gratis: Colab (`local[*]`) o Databricks Free Edition *(comprueba el nombre vigente)*

7.6 · Equivalencias

	pandas	PySpark
filtrar	`df[df.x == 1]`	`df.filter(F.col("x") == 1)`
agrupar	`groupby`	`groupBy`
unir	`merge`	`join`

7.7 · Orquestación

- **DAG:** tareas + dependencias, sin ciclos
- Calendario, **reintentos**, historial, alertas, **backfill**
- Airflow · Dagster · Prefect

```
extraer → bronce → plata → oro
                      └──→ calidad ─→ informe
```

7.7 · Idempotencia

> Ejecutar dos veces = ejecutar una

- borrar e insertar por partición
- `MERGE` por clave
- fecha **lógica**, no «hoy»

Sin ella, cada reintento duplica datos

7.8 · Kafka

- **Topic** → particiones → mensajes con **offset**
- **Grupos** de consumidores se reparten las particiones
- Entrega **al menos una vez** → hay **duplicados**
- Confirma el offset *después* de procesar

7.8 · Ventanas y marca de agua

- **Tiempo de evento** ≠ tiempo de llegada
- **Tumbling** de 5 min
- **Watermark** = máx. tiempo visto – retraso permitido
- Evento **tardío**: descartar, salida lateral o corregir por lotes

Más retraso permitido = más exactitud, más latencia

Caso práctico

Repositorio «pipeline de datos»:

1. ≥ 1 M filas en Parquet particionado
2. Capas + estrella + día idempotente
3. dbt: 3 modelos, 5 tests
4. Benchmark honesto
5. DAG con reintentos y backfill
6. README con diagrama

Llévate esto

1. **Mide** antes de escalar
2. Parquet + particiones por lo que filtras
3. Capas y **grano** claros
4. **Tests de datos** en cada ejecución
5. Todo **idempotente**
6. Empieza por lotes; streaming solo si hace falta